

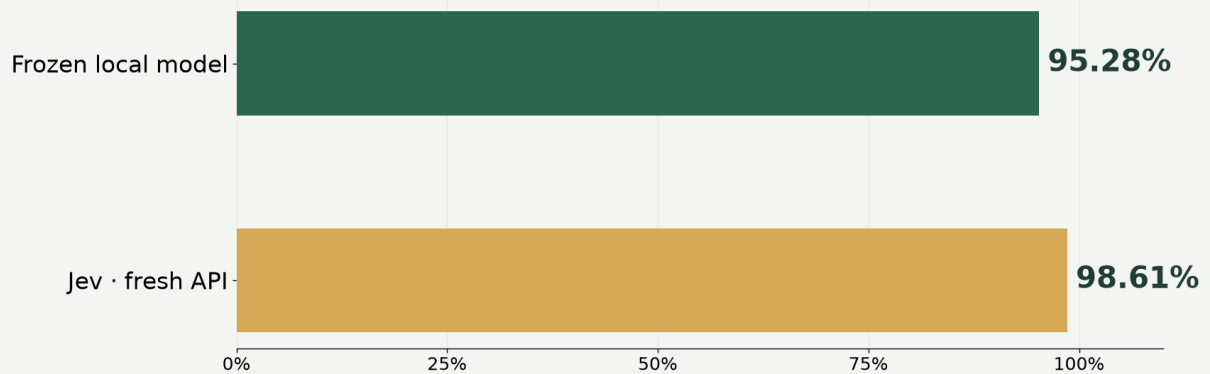
Can Astra Build Its Own Jev?

FILMING GUIDE / 21 SEPTEMBER 2026

The photo-aware local agency, the frozen text challenger, and the measured results.

The stronger model still lost.

360 fresh scenarios · 1,440 policy decisions per engine · zero request failures



Local minus Jev: -3.33 percentage points | 95% interval: -4.38 to -2.29

Synthetic, agent-reviewed references. Two writer-assisted adjudications; no human validation.

Source: round1-deeper-fp16 paired report · frozen before generation · all prespecified cases retained

V2 runs the demo. It did not beat Jev.

Same 360 fresh synthetic travel scenarios: first model 60.28%, V2 95.28%, Jev 98.61%. The app runs V2 under the app rule (quality gates passed, clearly beats V1), not as a Jev win. Reference repairs and two writer-assisted adjudications are disclosed; no human validation.

1,751 spoken words. 11.3 minutes at 155 WPM, with a planned 14.2-minute runtime including visual holds. Script-only narration source and a separate audio guide are supplied.

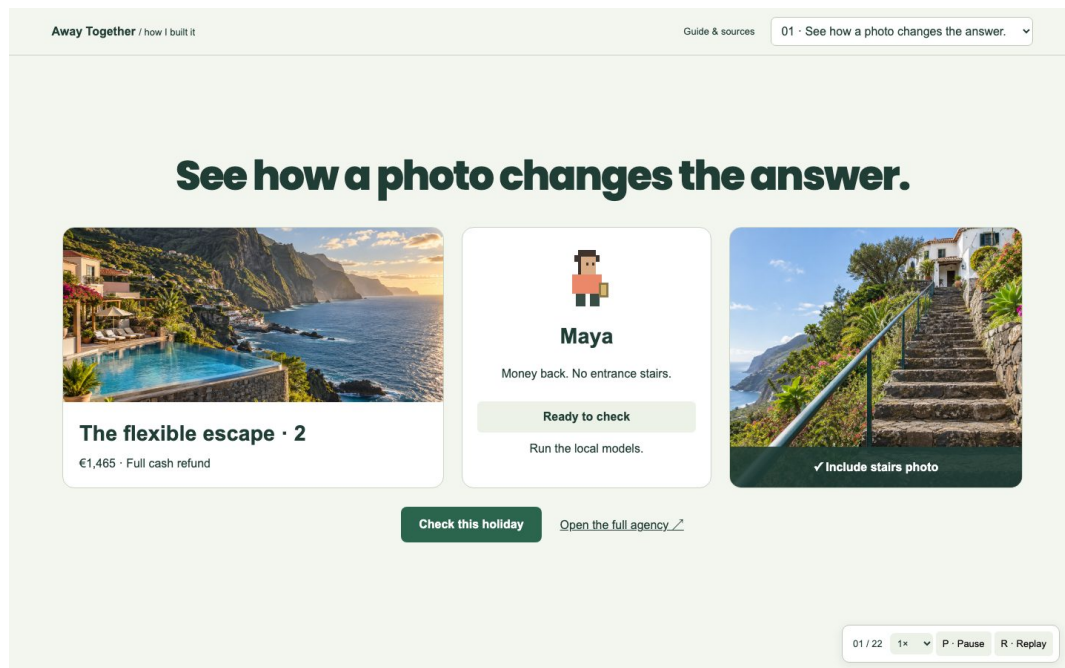
How to use this guide

Each scene page shows what is on screen, then exactly what to say. Screenshots come from the live filming site at 1440 x 900.

EDITING NOTE FOR EVERY SCENE

Let the result finish before speaking its meaning. P pauses diagrams; R resets current scene. Give the final visual time to read. Copy prompts into a coding assistant with local tools. When showing execution, keep actual results and disclose cut waiting time. Use the 1×, 1.5× or 2× controls for explanatory diagrams.

0:00 - 1:05 / See how a photo changes the answer



On screen: localhost:8770/#demo

SAY

I took the idea behind Jev and built a version for this travel job that runs entirely on my computer.

It even understands images, which Jev's current model doesn't support.

Then I handed it to Claude Opus 5.5, and it made it faster and more accurate.

Now imagine you run a travel agency.

You've got twelve customers, all with different budgets and wish lists.

One needs a pool.

Another arrives after midnight.

Maya wants a cash refund if she cancels, and an entrance without stairs.

Your job is to match each person with a holiday that fits.

The AI reads each holiday's fine print and looks at its photos.

Then the app checks what it found against every customer's wish list.

Watch what happens when we check this one.

PICTURE

Scene `#demo` at localhost:8770. Start with the live persona demo already visible. In the full agency, select The flexible escape · 2 (offer 10 of 40); the #demo scene already uses it. Deliver the Jev/local/images claim over the app. Run Maya with the stairs photo by 0:10; remove it and run again. Point at review by 0:25. Briefly show the full agency grid. If either request stalls, preserve real elapsed time and cut the wait transparently.

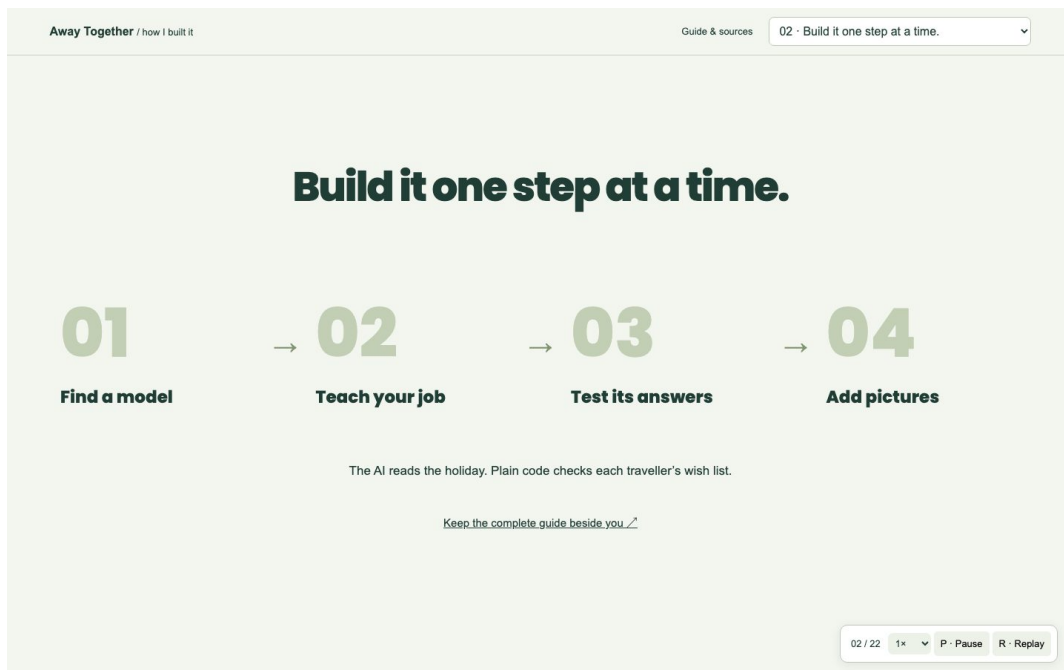
ON-SCREEN COPY

See how a photo changes the answer.

SOURCE / TRUTH CARD

Fresh /api/decide receipts; active V2 text model (models/active-model.json, app rule) plus pretrained vision. Fictional travellers and photos. TypeSafe model documentation checked 21 September 2026: current Jev text-only input; our image support is a separate pretrained integration. Opus 5.5 line (22 Sep 2026 session only): same-test accuracy 60.28% to 95.28% (experiments/v3/runs/v1-vs-v2-round1.json); first check after restart 8.5 s to about 0.12 s via startup warm-up; photo false alarms 10 to 4 on a held-out set (experiments/v3/vision/holdout). The original build was done with Codex; do not say Opus built it.

1:05 - 2:05 / Build it one step at a time



On screen: localhost:8770/#why

SAY

The written terms haven't changed.

The photo gave the system something else to consider.

Here's what's actually happening.

The AI never judges the travellers.

It only reads the holiday and answers a few simple questions about it.

Is there a cash refund?

Can you arrive after midnight?

Is the pool free?

Is a guided hike included?

Are there stairs at the entrance?

Then plain code compares those answers with each person's wish list and budget.

Now I'll show you how to build this for your own job.

Find a model, teach it with examples, test the answers, then connect pictures.

You can ask Claude to handle the downloads, setup and code.

Your part is explaining the job and checking whether its examples make sense.

I'll give you the project files and the prompts as we go.

PICTURE

Scene `#why` at localhost:8770. Point to the four-stage visual path.

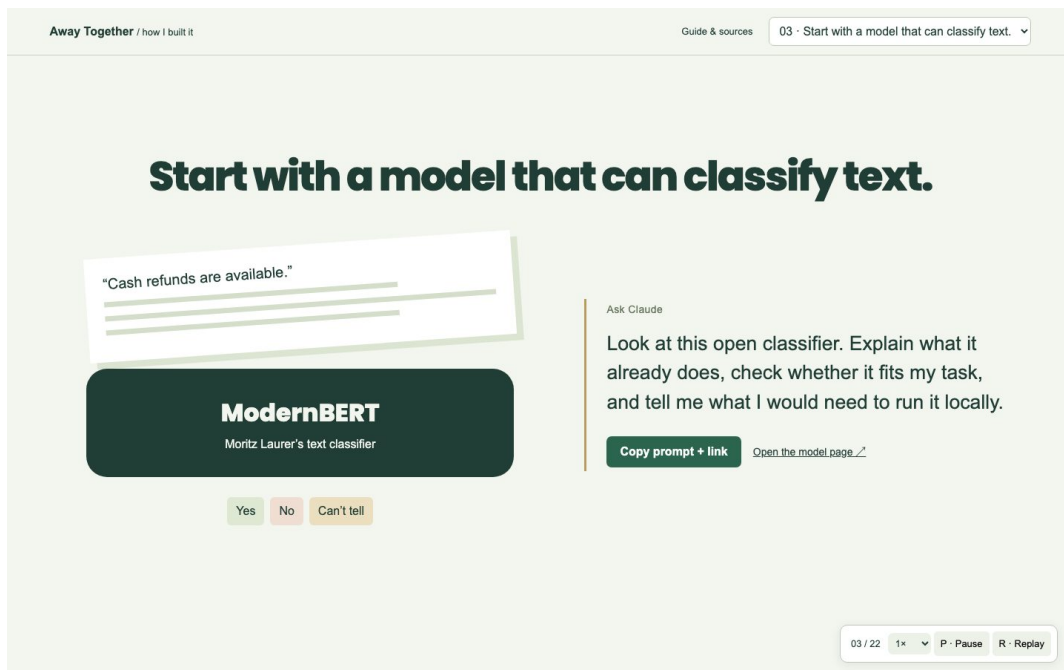
ON-SCREEN COPY

Build it one step at a time.

SOURCE / TRUTH CARD

docs/STEP-BY-STEP.md; exact tutorial commands and retained app.

2:05 - 2:50 / Start with a model that can classify text



On screen: localhost:8770/#choose-model

SAY

First, find a model that can already do the kind of job you need.

Hugging Face is where people share models you can download.

This is the one I started with, Moritz Laurer's ModernBERT model.

Someone has already trained it to compare text with possible answers.

We aren't starting from a blank model.

On the model page, check what it does, whether you can download it, its terms, and its results.

Save the exact version.

Claude can help you read the page, but a popular model still needs testing on your job.

PICTURE

Scene `#choose-model` at localhost:8770. Open the actual model card. Point to owner, task, weights and pinned revision; use the card as a navigation surface, not a fabricated screenshot.

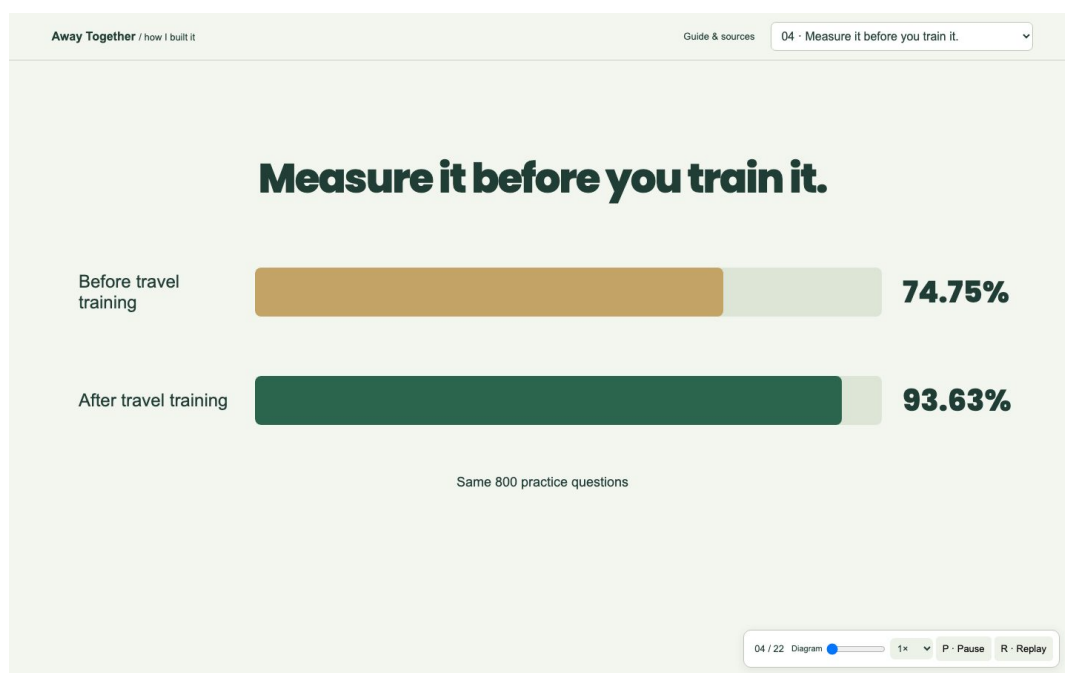
ON-SCREEN COPY

Start with a model that can classify text.

SOURCE / TRUTH CARD

Pinned Hugging Face author model card; travel_lab/nli.py NLI_ID and NLI_REV.

2:50 - 3:35 / Measure it before you train it



On screen: localhost:8770/#baseline

SAY

Before changing it, find out what it can already do.

On our practice questions, the starting model got about seventy-five percent right.

After we trained it on travel examples, it got about ninety-four percent on those same questions.

So training helped.

But we used those practice questions to pick the best version, so that score flatters it.

On a final exam it had never seen, that first model only got sixty-six point five percent.

Keep those two scores separate.

Test your own model before training, then check the improvement on questions it's never seen.

PICTURE

Scene `#baseline` at localhost:8770. Point at the before bar first, then after. Keep development and separate final-test caveat visible. Open the JSON receipt.

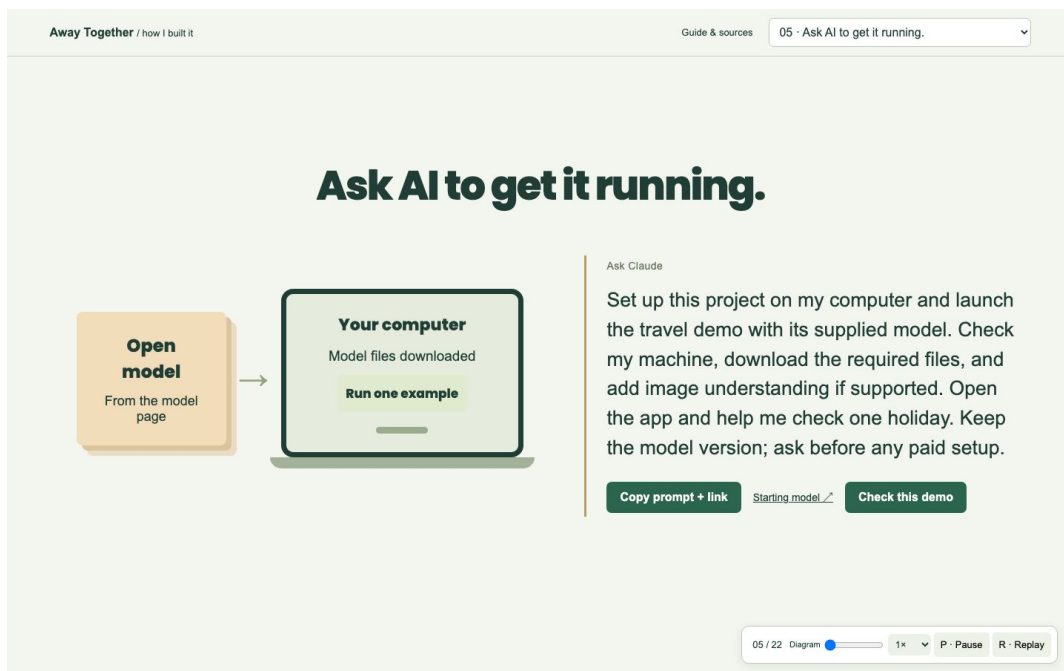
ON-SCREEN COPY

Measure it before you train it.

SOURCE / TRUTH CARD

models/selection.json baseline_dev and dev; runs/nli-baseline-dev.json; historical final summary. 800 development judgments.

3:35 - 4:20 / Ask AI to get it running



On screen: localhost:8770/#download

SAY

Give Claude the project and ask it to get the travel demo running on your computer. It can check your machine, download the required files, and open the app. Start with the supplied model, so you don't have to train anything to try it. Ask it to add the photo checker if your machine supports it. Have it show the model version and run one real holiday check. If your computer can't run the setup, ask what's missing before going further. Now you've got a working example to learn from.

PICTURE

Scene `#download` at localhost:8770. Copy the setup prompt and repository link into Claude Code. Explain the file-to-computer animation, then open the actual agency after setup. The starting model link is available beside the prompt.

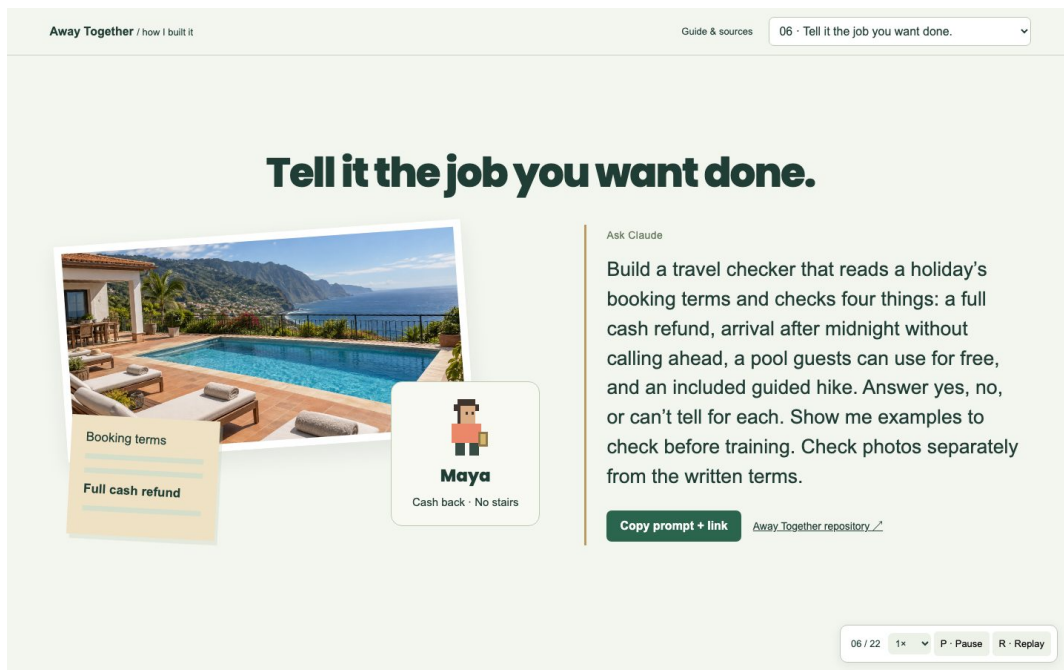
ON-SCREEN COPY

Ask AI to get it running.

SOURCE / TRUTH CARD

tools/tutorial.py download; live pinned snapshot_download receipt; docs/STEP-BY-STEP.md. tools/first_run.py; docs/SETUP.md; supplied model versus new tutorial checkpoint.

4:20 - 5:10 / Tell it the job you want done



On screen: localhost:8770/#build

SAY

Tell it exactly what a good match means.

Our checker reads a holiday's booking terms and looks for four things.

A full cash refund, arriving after midnight without calling ahead, a pool you can use for free, and a guided hike that's included.

Hotel credit isn't a cash refund.

A pool in a photo doesn't prove it's free to use.

Every question gets one of three answers.

Yes, no, or can't tell.

Maya also wants no stairs, and a separate model checks the photos for that.

Ask the AI to show you a few examples before it writes the ones it'll learn from.

You decide whether those examples follow your rules.

PICTURE

Scene `#build` at localhost:8770. Show the holiday, booking terms and Maya beside the brief. Walk through its four concrete requirements.

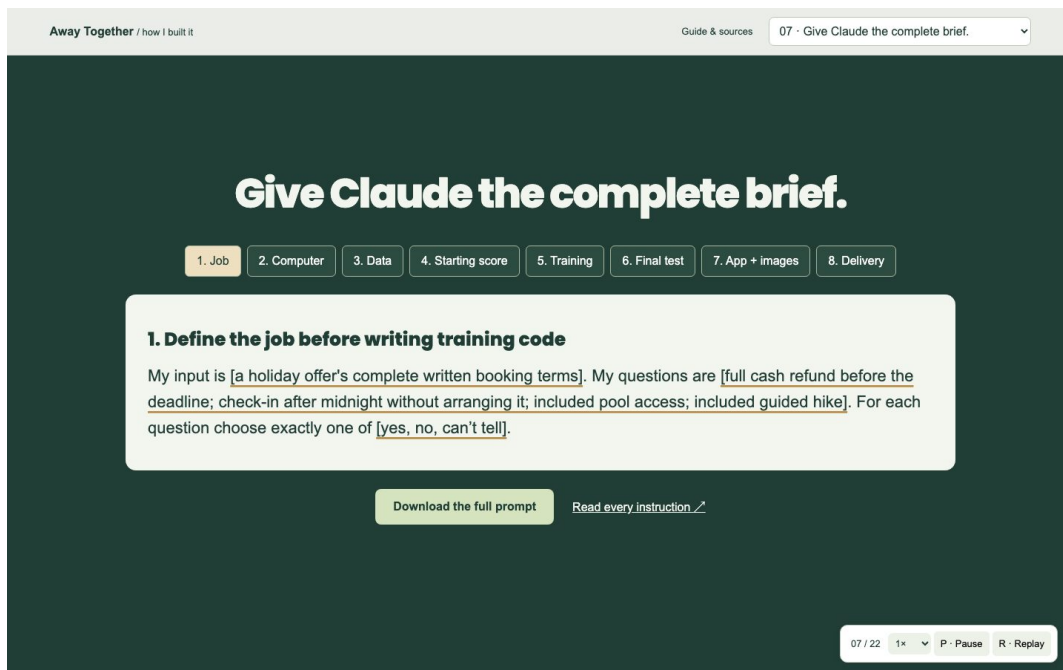
ON-SCREEN COPY

Tell it the job you want done.

SOURCE / TRUTH CARD

prompts/TRAIN-MY-SPECIALIST.md; supplied travel hypotheses and task data. travel_lab/nli.py HYPOTHESES; separate visual requirements.

5:10 - 6:05 / Give Claude the complete brief



On screen: localhost:8770/#prompt

SAY

Here's the complete prompt.

It's long because it asks for more than a model.

The first section describes your job.

Change the parts in brackets.

The data section tells Claude to show you examples and keep related documents together.

The training section asks for a tiny test run before the full one.

The final-test section says to lock the model and keep every mistake.

Keep those steps when you use it for a different job.

Download the whole prompt, paste it into Claude with the project open, and have it walk you through each step.

It's a template you can reuse.

My own project still took follow-ups and fixes.

PICTURE

Scene `#prompt` at localhost:8770. Click Job, Data, Training and Final test tabs. Open the complete prompt page and show that the download has all eight sections. Underlines identify editable task fields.

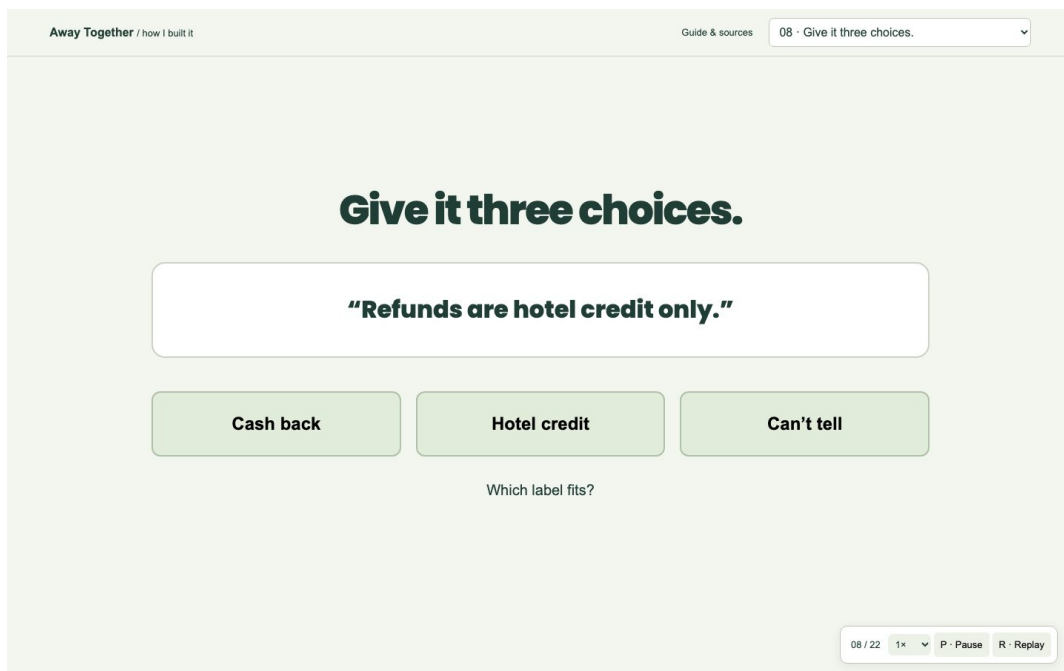
ON-SCREEN COPY

Give Claude the complete brief.

SOURCE / TRUTH CARD

Exact downloadable eight-part prompt; prepared specification, not historical chat.

6:05 - 6:35 / Give it three choices



On screen: localhost:8770/#labels

SAY

This policy offers hotel credit.

Maya wants a cash refund, so it fails her requirement.

A clear cash refund passes.

If the policy doesn't say, we can't tell.

Those are our three labels.

The model picks one of them.

That gives our app a short answer it can use.

It can still pick the wrong one.

PICTURE

Scene `#labels` at localhost:8770. Click Cash back, then Hotel credit. Point to the mapping between the teaching labels and actual output labels.

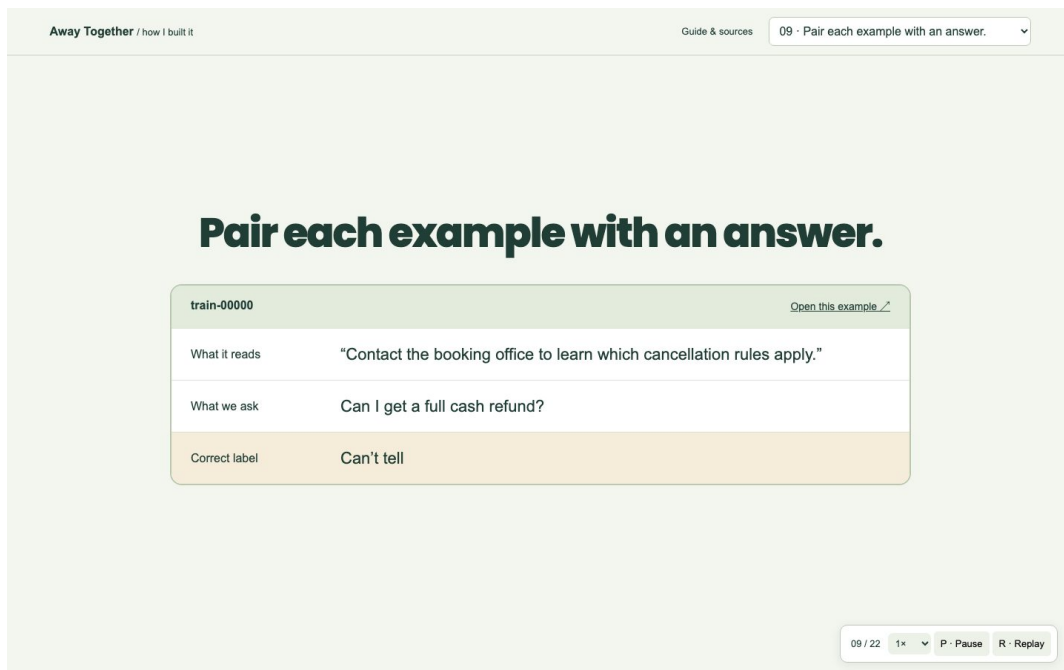
ON-SCREEN COPY

Give it three choices.

SOURCE / TRUTH CARD

Teaching exercise; actual NLI labels in data/train.jsonl. Fixed format is not a deterministic-output claim.

6:35 - 7:10 / Pair each example with an answer



On screen: localhost:8770/#training

SAY

We started with ModernBERT, a model that already understands language patterns.

To make it better at this job, we gave it travel examples.

Each has three parts.

What it reads, what we ask, and the checked answer.

This actual record says to contact the booking office.

So we can't tell whether there's a cash refund.

Our examples are made up.

For your job, check every answer carefully.

Wrong answers teach the wrong lesson.

PICTURE

Scene `#training` at localhost:8770. Open actual train-00000 excerpt. Download the full record. Point at input, question, answer; show the separate held-out file only as data provenance.

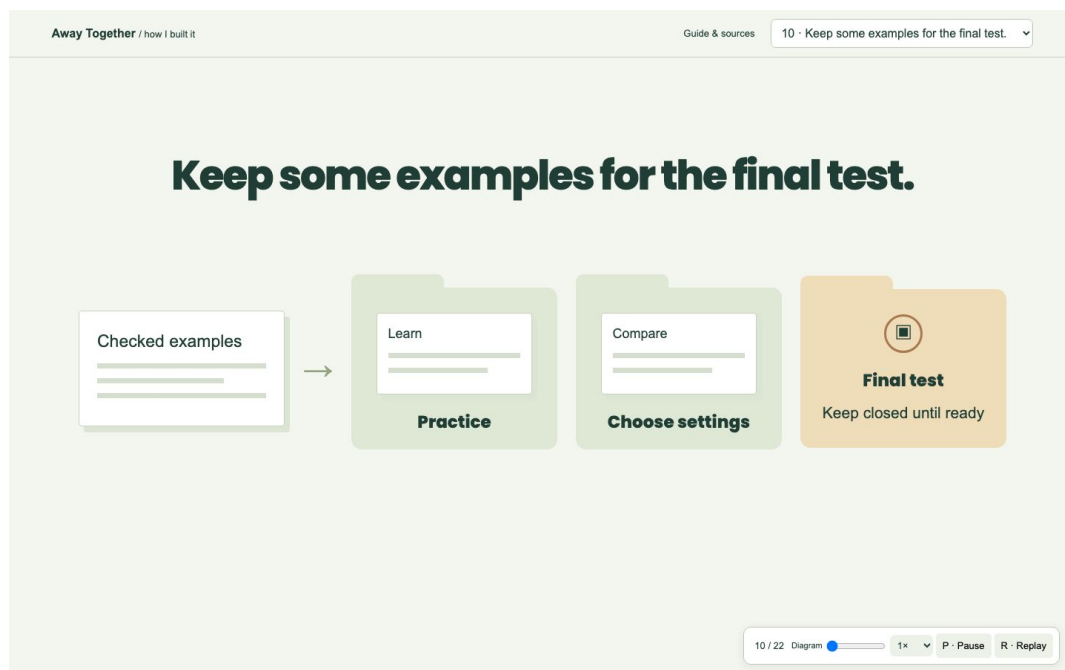
ON-SCREEN COPY

Pair each example with an answer.

SOURCE / TRUTH CARD

assets/training-example.json exact data/train.jsonl record; synthetic, no human labels.

7:10 - 7:45 / Keep some examples for the final test



On screen: localhost:8770/#data-split

SAY

Keep three groups of examples.

These teach the model.

These help you choose the settings.

And these stay closed until the end, so you can see how it handles something new.

Ask your coding assistant to make that split and keep related examples together.

Two copies of the same customer conversation should never end up on opposite sides.

You don't need to remember file extensions.

You need to understand why the final group stays separate.

PICTURE

Scene `#data-split` at localhost:8770. Play the practice, settings and sealed-test folders appearing in order.

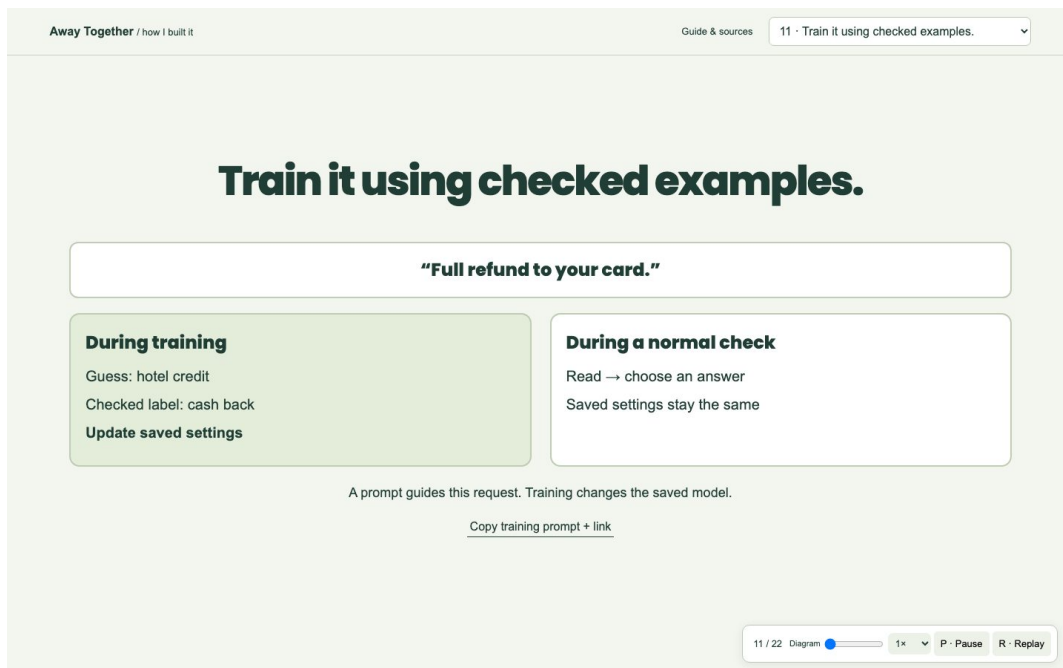
ON-SCREEN COPY

Keep some examples for the final test.

SOURCE / TRUTH CARD

Tutorial prepare validation; travel data provenance and split design.

7:45 - 8:20 / Train it using checked examples



On screen: localhost:8770/#practice

SAY

The model guesses.

Training compares that guess with your checked answer and adjusts the model's saved settings.

That's fine-tuning.

Ask Claude to test the original model first, run a tiny training check, then train on your examples and save the best version.

It should show you the results as it goes.

You check what the examples mean; the coding assistant runs the training.

Using the finished model later doesn't train it again.

PICTURE

Scene `#practice` at localhost:8770. Play the existing training animation. Copy the training prompt below it when moving to execution.

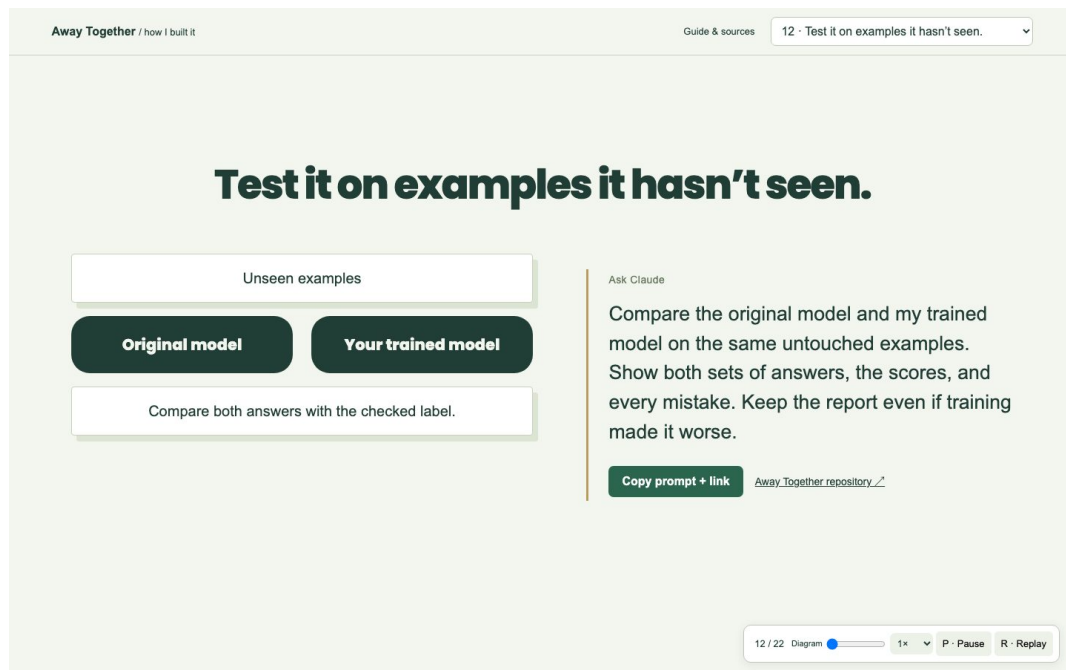
ON-SCREEN COPY

Train it using checked examples.

SOURCE / TRUTH CARD

Training code and saved checkpoints. Diagram is conceptual, not a measured individual training step.

8:20 - 8:55 / Test it on examples it hasn't seen



On screen: localhost:8770/#test

SAY

Now ask it to compare the original model and your trained model on exactly the same unseen examples.

Show both sets of answers and every mistake.

Accuracy tells you how often it matched the checked answers.

Don't just look at the average.

Open the cases it got wrong.

And if training made it worse, keep that result.

Those mistakes can help your next experiment, but you can't call the same test new again.

PICTURE

Scene `#test` at localhost:8770. Play identical unseen examples flowing to both models. Copy the test prompt.

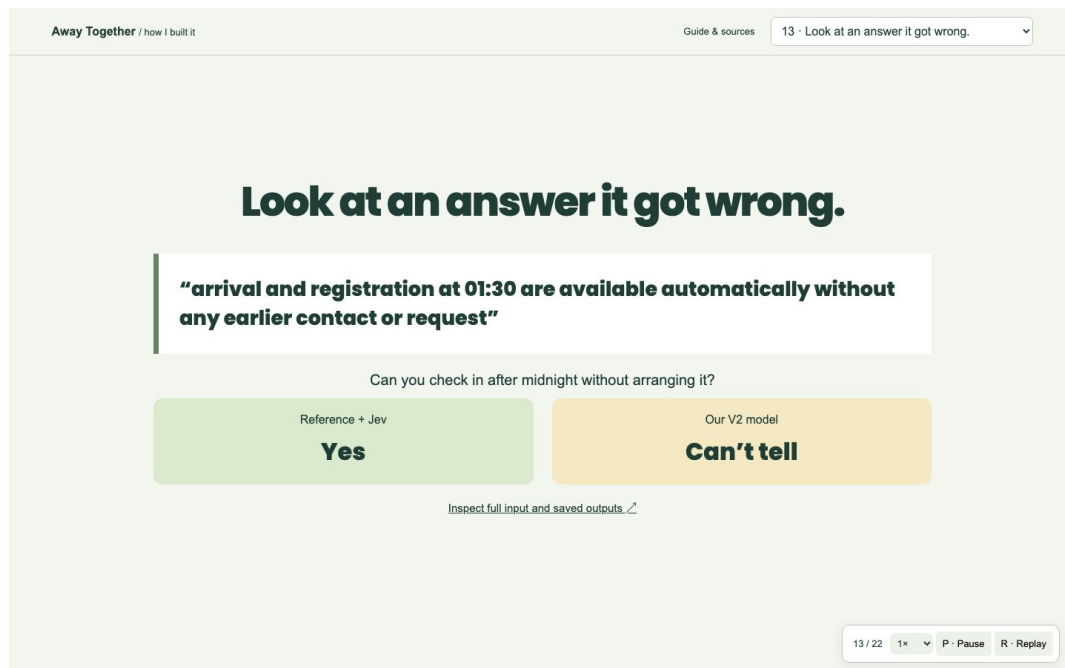
ON-SCREEN COPY

Test it on examples it hasn't seen.

SOURCE / TRUTH CARD

Frozen V2 protocol; 52 document repairs; two writer-assisted adjudications; hashes. V1 held-out download is labeled separately.

8:55 - 9:25 / Look at an answer it got wrong



On screen: localhost:8770/#failure

SAY

Here's a real mistake from that test.
Arrival at one thirty in the morning is allowed without calling ahead.
The reference says yes.
Jev says yes.
My later model says it can't tell.
That sends a clear case to unnecessary review.
Both received the same full document.
We're looking at an excerpt.
This is why I want you to inspect the mistakes too.

PICTURE

Scene `#failure` at localhost:8770. Show case ...0036 arrival excerpt and both saved outputs. Open full JSON to show the input is longer than the excerpt.

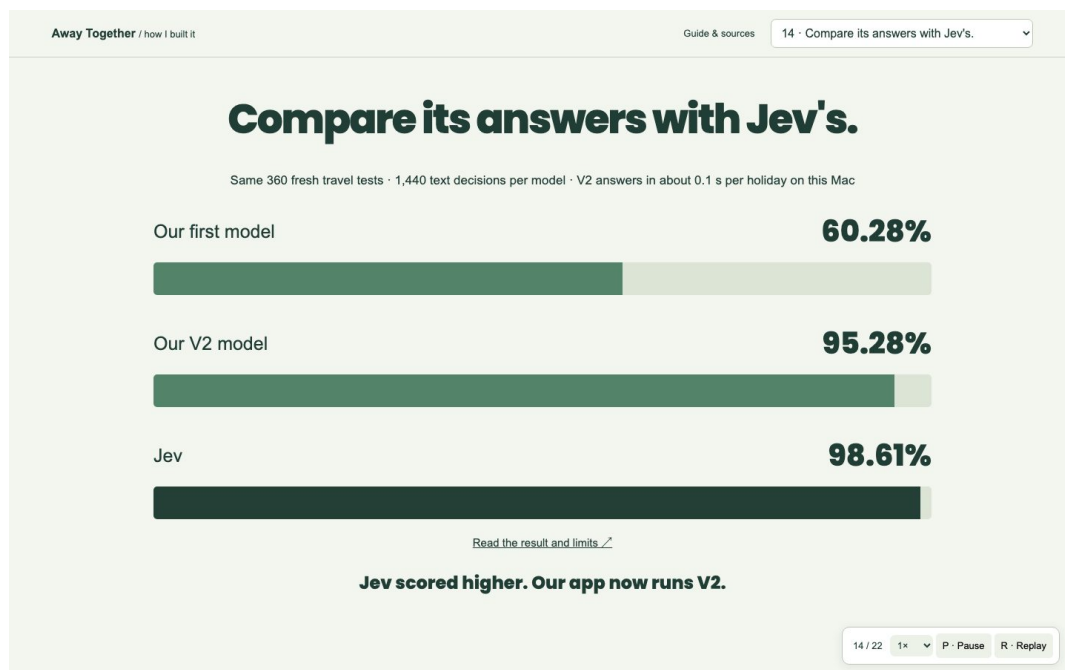
ON-SCREEN COPY

Look at an answer it got wrong.

SOURCE / TRUTH CARD

assets/benchmark-case.json; raw frozen local and Jev rows for round1-deeper-fp16-0036, arrival.

9:25 - 9:55 / Compare its answers with Jev's



On screen: localhost:8770/#results

SAY

That later model agreed with our answer key about ninety-five percent of the time.

Jev was closer to ninety-nine.

I didn't beat Jev.

On that same test, my first model only got about sixty percent.

So the demo you saw runs the later model, right here on this computer.

And this chart tests text, not picture understanding.

We built the workflow.

Its reliability still needs work.

PICTURE

Scene `#results` at localhost:8770. Show three common-axis bars from the same 360 fresh scenarios: first model 60.28%, V2 95.28%, Jev 98.61%. Point to text-only scope. The first-model bar is a same-test comparison; its separate 66.5% test stays out of this chart.

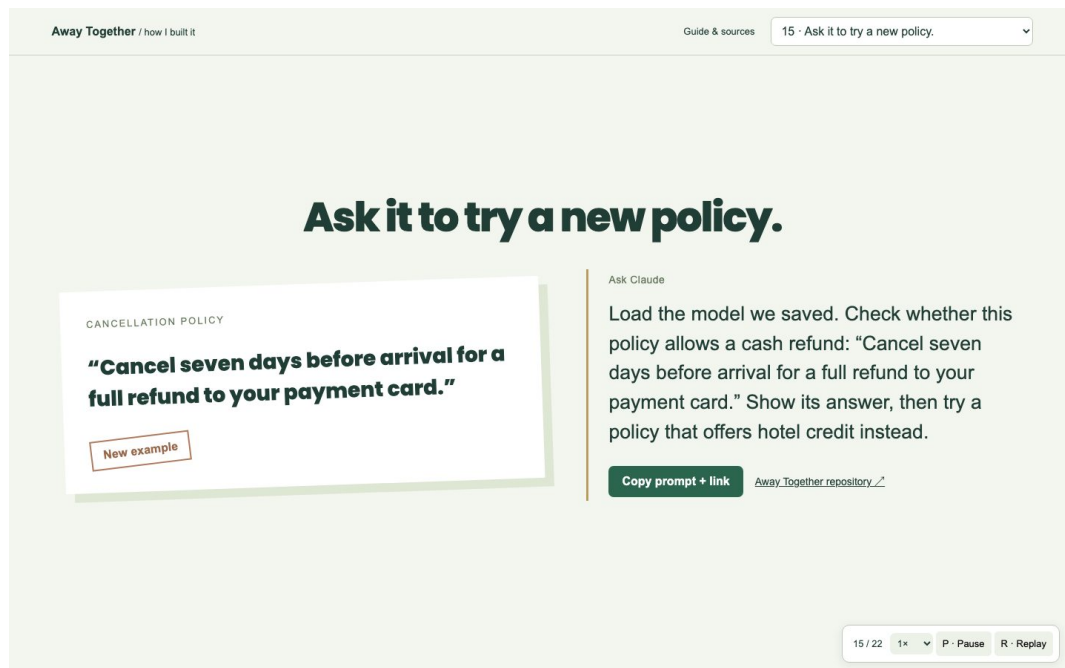
ON-SCREEN COPY

Compare its answers with Jev's.

SOURCE / TRUTH CARD

FROZEN-V2-RESULTS.md; qualification-decision.json; experiments/v3/runs/v1-vs-v2-round1.json (both models frozen before the corpus, no tuning); models/active-model.json basis app-quality-gates.

9:55 - 10:35 / Ask it to try a new policy



On screen: localhost:8770/#predict

SAY

Here's a new policy.

It offers a full refund to the payment card.

Ask your coding assistant to load the model you saved and run this policy through it.

The saved model should make the decision.

Then change the policy to hotel credit and try again.

That gives you a simple way to check whether it's distinguishing the thing you care about.

Keep the output visible so you can inspect the answer.

PICTURE

Scene `#predict` at localhost:8770. Show the cancellation letter and prompt. Paste into a coding assistant for actual saved-model inference; do not substitute the assistant's own answer.

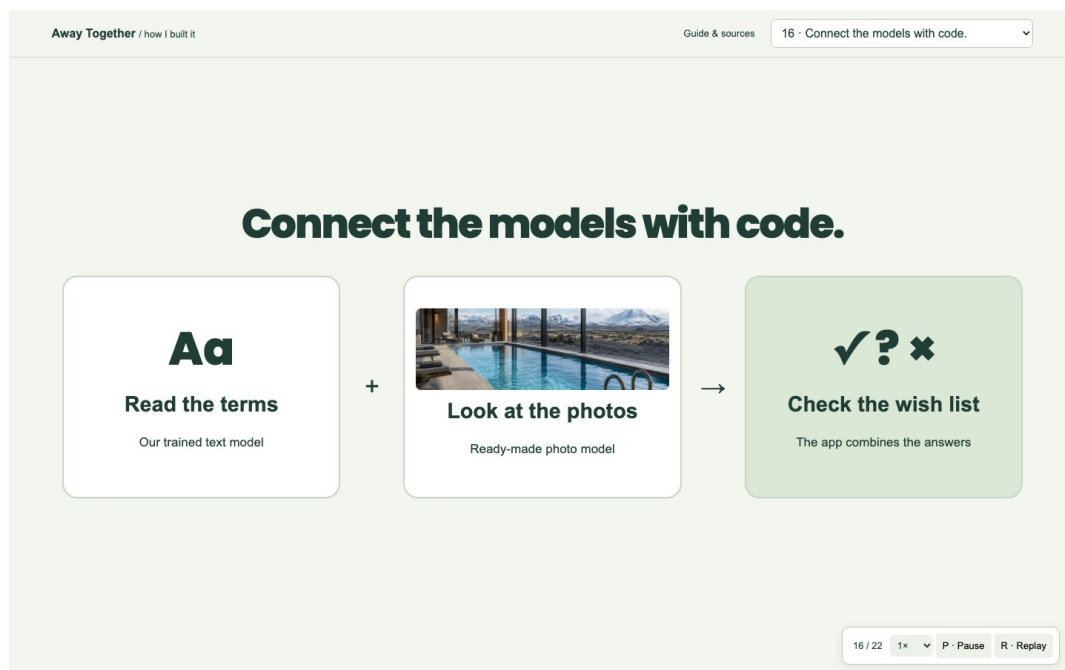
ON-SCREEN COPY

Ask it to try a new policy.

SOURCE / TRUTH CARD

tools/tutorial.py predict; downloadable JSON; actual smoke prediction receipt.

10:35 - 11:05 / Connect the models with code



On screen: localhost:8770/#map

SAY

There are three pieces.

A model reads the terms.

Another looks at the pictures.

Code checks those answers against Maya's wishes and budget.

I trained the text reader for travel and connected an already trained image reader.

Multimodal means we're using words and pictures.

Price comparisons stay in ordinary code.

You don't need AI to tell you whether twelve hundred is more than a thousand.

PICTURE

Scene `#map` at localhost:8770. Point to text reader, pretrained photo reader and app rules in order; keep Maya visible when returning to the agency.

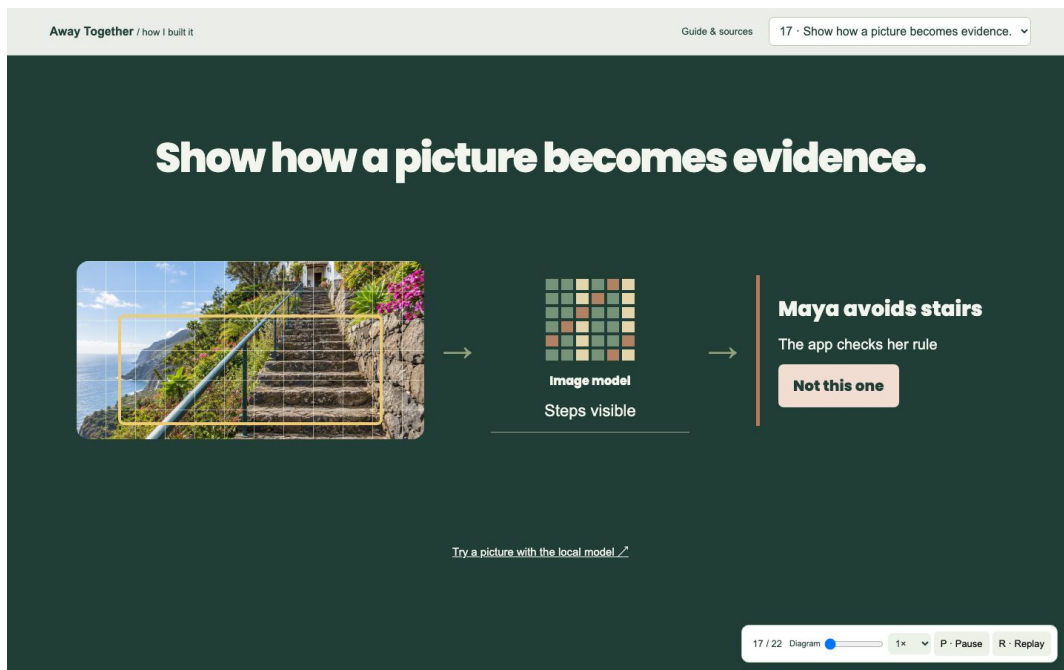
ON-SCREEN COPY

Connect the models with code.

SOURCE / TRUTH CARD

travel_lab/vision.py; catalogue.py; active selection.json.

11:05 - 11:45 / Show how a picture becomes evidence



On screen: localhost:8770/#vision

SAY

Here's how pictures join the decision.

The actual image goes into a separate model that already knows how to read pictures.

We ask about visible features, like entrance steps.

It returns those observations, and our app compares them with Maya's requirements.

I connected this image model, which someone else already trained, through OpenJev.

I didn't train the text model to see.

Ask the coding assistant to connect the photo checker, then test it with a real image and with the image missing.

PICTURE

Scene `#vision` at localhost:8770. Play photo, pixel grid, feature answers and Maya's requirement in order. The grid illustrates pixel input, not an exact architecture trace. Open photo lab for fresh inference.

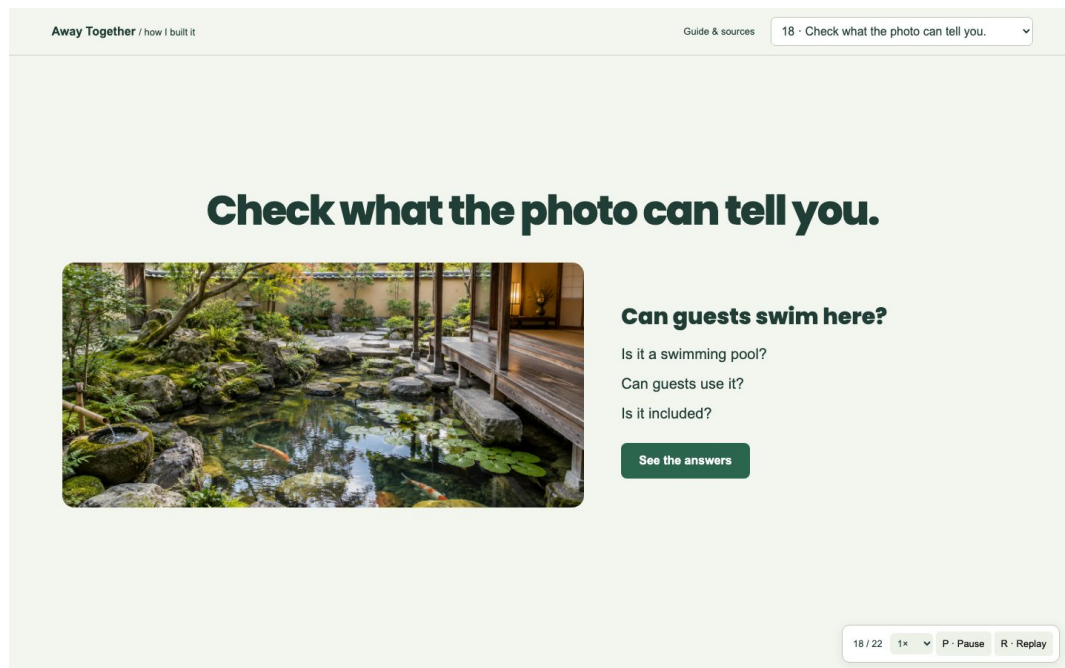
ON-SCREEN COPY

Show how a picture becomes evidence.

SOURCE / TRUTH CARD

Fresh /api/observe receipt; fixed loopback endpoint; pretrained DiffusionGemma. Generated fictional catalogue photos.

11:45 - 12:15 / Check what the photo can tell you



On screen: localhost:8770/#boundary

SAY

There's water here, but that doesn't prove it's a swimming pool.

Even a pool photo can't tell us whether access costs extra.

Pictures show visible features.

Terms tell us what's promised.

Our forty offers share nine fictional images, so this isn't a test of forty real hotels.

And removing Maya's stairs photo never proves the entrance is accessible.

PICTURE

Scene `#boundary` at localhost:8770. Show pond, ask the three questions and reveal the conclusion. Refer back to the missing stairs photo.

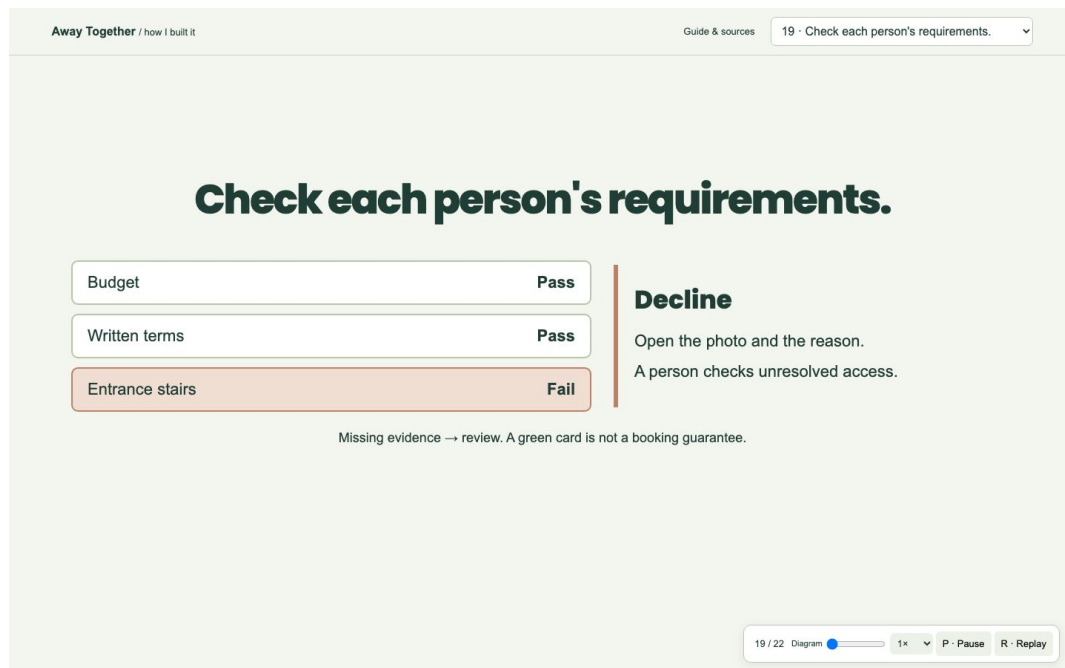
ON-SCREEN COPY

Check what the photo can tell you.

SOURCE / TRUTH CARD

Generated-image ledger; location-photos.json; image evidence limitations.

12:15 - 12:45 / Check each person's requirements



On screen: localhost:8770/#rules

SAY

The app combines the checks.

Budget passes.

Terms pass.

Stairs fail, so Maya gets a decline.

Missing evidence goes to review unless another requirement already fails.

Open her card and inspect the reason.

A green card means these checks passed according to our models.

A person still confirms the booking details.

The app doesn't book anything.

PICTURE

Scene `#rules` at localhost:8770. Play budget/terms/photo gates. In the actual agency open a traveller drawer and show the clause/photo that caused decline or review.

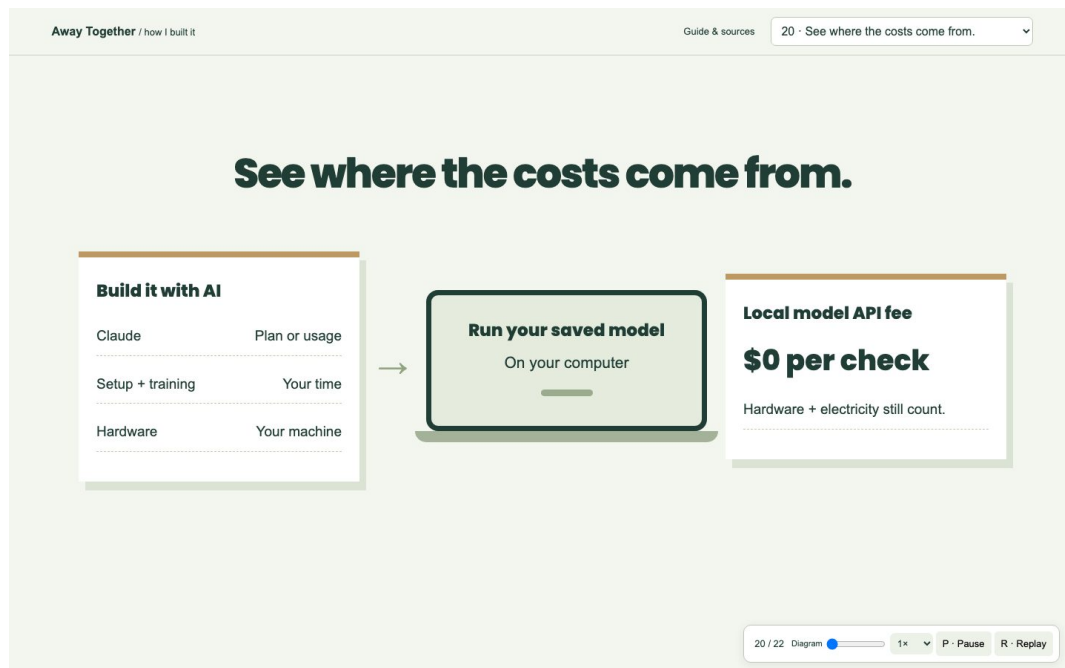
ON-SCREEN COPY

Check each person's requirements.

SOURCE / TRUTH CARD

catalogue.verdict; apply_visual_requirements; live reason drawer; companion local API example.

12:45 - 13:20 / See where the costs come from



On screen: localhost:8770/#cost

SAY

There are two different costs here.

Claude helps you build it, and your plan or usage can cost money.

There's also setup, training and your computer.

Once the saved model runs on your computer, you don't pay an AI company for each check.

You still pay for the computer and the electricity.

So this isn't a claim that the entire project cost nothing.

Measure the job you actually want to run before you decide what it saves.

PICTURE

Scene `#cost` at localhost:8770. Animate the build receipt, local computer and per-check model API fee. No fabricated total or savings percentage.

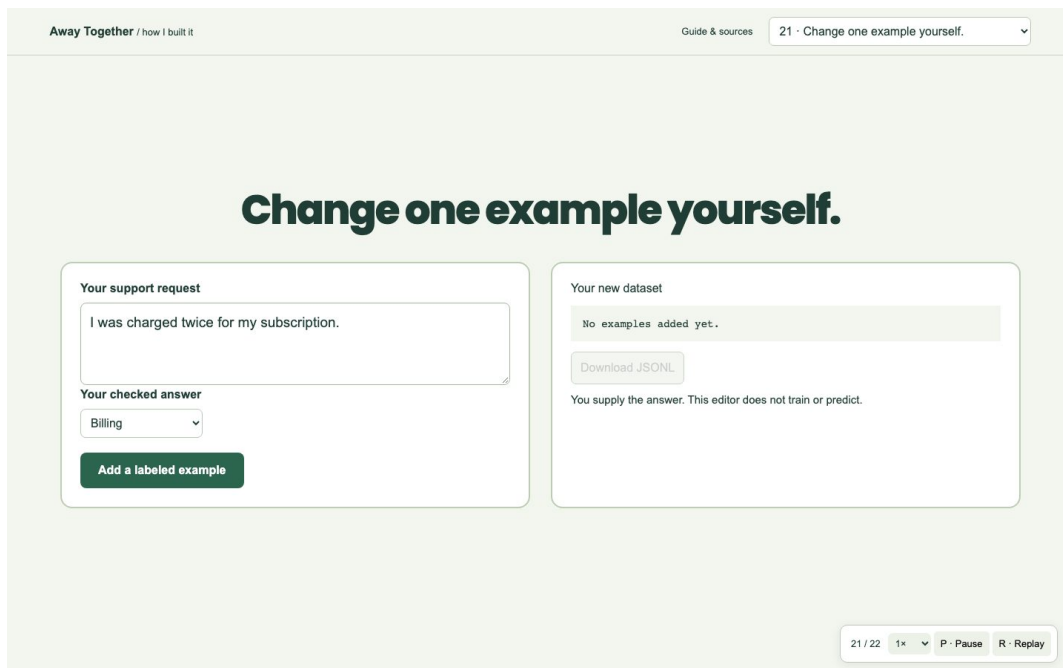
ON-SCREEN COPY

See where the costs come from.

SOURCE / TRUTH CARD

V2 app timing receipts (experiments/v3/runs/v1-vs-v2-round1.json) and photo integration scan; no measured total-cost ratio.

13:20 - 13:55 / Change one example yourself



On screen: localhost:8770/#repurpose

SAY

Say your job is sorting support messages.

Here's someone charged twice.

Choose Billing, add the example, and download the data file.

You've made a checked example.

You haven't trained a support model yet.

The guide shows how to prepare the data, change the task and keep a separate test.

Start with cases you understand, including confusing ones.

Changing the labels alone won't teach the travel model a new job.

PICTURE

Scene `#repurpose` at localhost:8770. Edit support request, choose its human label, add it and download JSONL. Run `prepare_task.py` on the saved sample. Show held-out separation in the adaptation guide.

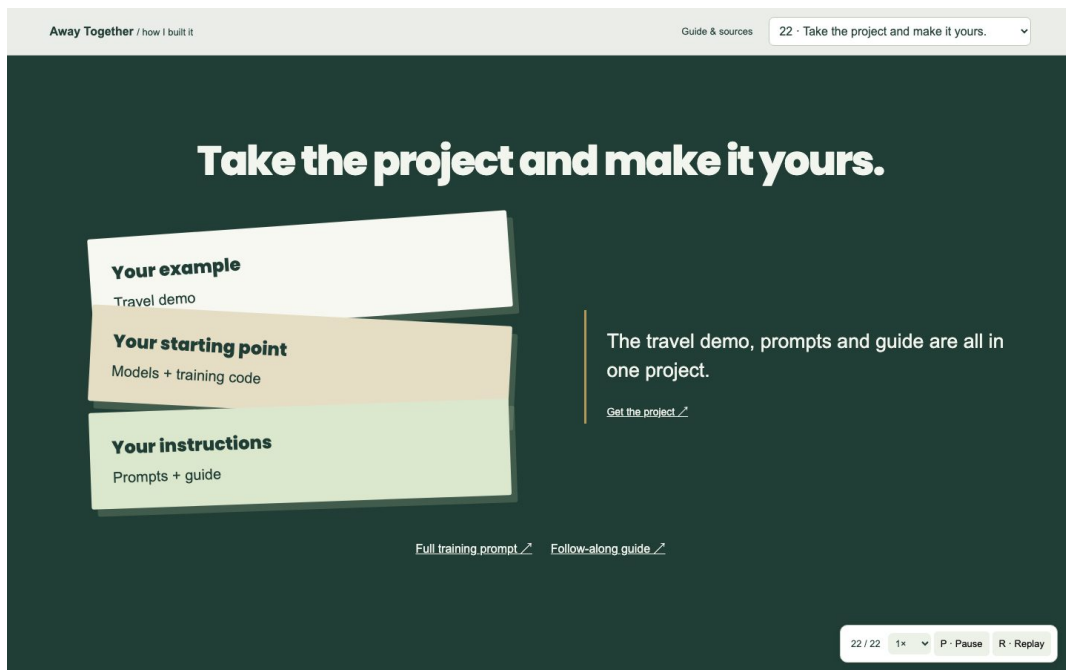
ON-SCREEN COPY

Change one example yourself.

SOURCE / TRUTH CARD

Real JSONL export; `tools/prepare_task.py`; `docs/ADAPT-YOUR-OWN.md`. No completed support-model claim.

13:55 - 14:15 / Take the project and make it yours



On screen: localhost:8770/#take-it

SAY

The travel demo, prompts and guide are all in one project.

Take them and adapt them to a job you understand.

Let AI handle the code.

You check whether the answers make sense.

PICTURE

Scene `#take-it` at localhost:8770. Show the source kit and repository link. End without repeating setup or training steps.

ON-SCREEN COPY

Take the project and make it yours.

SOURCE / TRUTH CARD

One repository stays private until the video goes live. Verify signed-out source and model-download access before adding the description link. Original-code reuse terms still need a decision. Retain all upstream credits.

Before the take

Start the OpenJev vision service on localhost:8081 and the agency on localhost:8765. Reset with R. Select Four friends. Choose The flexible escape · 2 (holiday 10 of 40) and run it fresh. P pauses or resumes the presentation. Escape closes details. The agency warms both models when it starts; wait about 20 seconds after launch before the first take.

The charts show saved benchmark results. The agency serves V2; the results chart compares it with the first model and Jev on the same test. Photos, travellers and offers are fictional. Source-code linkage is structured application logic.

Truth boundaries

Final references are synthetic and agent-reviewed, with 52 document repairs and two non-unanimous writer-assisted adjudications. All 360 scenarios remain. No human validation. The public workflow lane measures teacher agreement; upstream exposure may apply. Thirty-five workflow decisions failed the frozen context limit and remain in the denominator. Official TypeSafe benchmark scores were not reproduced.

Measured agency timing

Active V1: cold first HTTP request 10.1 seconds including loading. Three fresh forty-offer scans took 4.0, 4.2 and 4.5 seconds. Warm HTTP median 105 ms, 121 unique fresh requests, zero failures. These are text-only historical timings, separate from V2 model-only latency. The photo-aware 40-offer integration scan took 40.7 seconds in its recorded run.

Companion resource

Show the verified local bundle, including both checkpoints and original audit evidence. Add a public download link only once separately arranged. The OpenJev image branch ran locally; its pinned model is downloaded separately. The earlier text-only diffusion comparison was not executed. No public upload, deployment or messaging is implied.